Задача 1. Вычисление экспоненты

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В данной задаче нужно научиться вычислять экспоненту от заданного числа X, то есть e^X для e — натурального логарифма. Использовать функции exp, pow и подобные из стандартной библиотеки или откуда-то ещё **запрещено.** Лучше вообще math.h не подключать.
Подсказка: используйте ряд Тейлора.

Формат входного файла

В первой строке записано одно целое число N — количество аргументов, для которых нужно вычислить экспоненту ($1 \le N \le 10^4$). Далее идёт N строк, по одному вещественному числу X в каждой. Каждое число — это число, экспоненту от которого надо вычислить. Все числа X лежат в диапазоне $(-100, 100)$.

Формат выходного файла

Выведите N строк, в каждой строке одно вещественное число, которое равно $\exp(X) = e^X$ для соответствующего аргумента X из входного файла.
Следует выводить числа с помощью формата "%0.15g", чтобы число выводилось в экспоненциальном виде с 12 знаками после десятичной точки. Ответ считается верным, если его относительная ошибка не превышает 10^{-15}.

Пример

input.txt	output.txt
8 0.0 1.0 -1.0 2.0 -2.0 100.0 -100.0 0.189376476361643	1 2.71828182845905 0.367879441171442 7.38905609893065 0.135335283236613 2.6881171418161 4e+43 3.72007597602084e-44 1.20849583696666

Задача 2. Вычисление синуса

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В данной задаче нужно научиться вычислять синус. Использовать функцию sin из стандартной библиотеки или откуда-то ещё запрещено.

Подсказка: используйте ряд Тейлора.

Формат входного файла

В первой строке записано одно целое число N — количество аргументов, для которых нужно вычислить синус ($1 \le N \le 10^5$). Далее идёт N строк, по одному вещественному числу X в каждой. Каждое число — это число, синус которого надо вычислить.

Все числа X по абсолютной величине не превышают единицу (заданы в радианах).

Формат выходного файла

Выведите N строк, в каждой строке одно вещественное число, которое равно $\sin X$ для соответствующего аргумента X из входного файла.

Рекомендуется выводить числа с помощью формата "%0.15lf", чтобы выводилось 15 знаков после десятичной точки. Ошибка в каждом вашем ответе не должна превышать 10^{-15}.

Пример

input.txt	output.txt
5 0.0 0.523598775598298 0.785398163397448 1.047197551196597 1.570796326794896	0.000000000000000 0.500000000000000 0.707106781186548 0.866025403784439 1.000000000000000

Задача 3. Имена и возрасты

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Есть набор рисунков с подписями автором в стиле “Зоя 17 лет”. Нужно реализовать функцию для сбора статистики по именам и возрастам авторов.
Каждая подпись должна представляться структурой:

```
typedef struct Label_s {
    char name[16]; // имя автора (заканчивается нулём)
    int age;       // возраст автора (сколько лет)
} Label;
```

Каждая подпись должна представляться структурой:

```
typedef struct NameStats_s {
    int cntTotal; // сколько всего подписей
    int cntLong;  // сколько подписей с именами длиннее 10 букв
} NameStats;
```

Статистика возрастов должна представляться структурой:

```
typedef struct AgeStats_s {
    int cntTotal;  // сколько всего подписей
    int cntAdults; // сколько подписей взрослых (хотя бы 18 лет)
    int cntKids;   // сколько подписей детей (меньше 14 лет)
} AgeStats;
```

Функция для вычисления статистик должна иметь сигнатуру:

```
void calcStats(const Label *arr, int cnt, NameStats *oNames, AgeStats *oAges);
```

Здесь oNames и oAges — адреса структур, в которые нужно записать результат.

Формат входного файла

В первой строке содержится целое число N — количество подписей в файле ($1 \le N \le 1\,000$). Каждая подпись записана в виде “[имя] [возраст] let”. Все имена не длиннее 15 символов, возрасты целые, положительные, не больше 5 000.

Формат выходного файла

Нужно вывести статистики NameStats и AgeStats в формате, приведённом в примере.

Пример

input.txt	output.txt
7 Zoya 17 let Kirill 5 let Ivan 1 let Vasiliy 15 let Tutankhamun 3360 let Innokentiy 21 let Dozdraperma 70 let	names: total = 7 names: long = 2 ages: total = 7 ages: adult = 3 ages: kid = 2

Задача 4. Угол в треугольнике

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется найти угол BAC в заданном треугольнике $\angle ABC$.

Формат входного файла

В первой строке задано целое число Q — количество тестовых случаев ($1 \le Q \le 3 \cdot 10^5$). В каждой из следующих Q строк описан один случай.

Каждый случай описывается шестью вещественными числами: Ax, Ay — координаты вершины угла, Bx, By — координаты другой вершины треугольника, Cx, Cy — координаты третьей вершины.

Все вещественные числа заданы с максимально возможной точностью и по абсолютной величине не превышают 10^3.

Пусть M — максимум из абсолютных величин всех шести координат, заданных в тестовом случае. Гарантируется, что $|AB|$, $|AC| > \frac{1}{20}M$. То есть стороны треугольника, инцидентные искомому углу, имеют довольно большую длину.

Формат выходного файла

Нужно вывести Q строк, в каждой из которых должен быть записан угол при вершине A в треугольнике ABC. Все углы нужно выводить с максимально возможной точностью, рекомендуется использовать формат "%0.20g". Углы нужно выводить в градусах, в пределах от 0 до 180 градусов включительно.

Ответ засчитывается, если он отличается от истинного менее чем на 10^{-11}.

Пример

input.txt	output.txt
8 2 1 2 3 5 5 2 1 4 3 2 3 3 1 3 5 2 3 0 0 1 0 10 0 0 0 1 0 -10 1 7 4 3 3 3 3 0 0 1 0 1 1e-5 0 0 1 0 1e-5 1	36.869897645844019962 45 26.565051177077990019 0 174.28940686250035697 0 0.00057295779511172474814 89.999427042204885652

Задача 5. Числа Фибоначчи

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Последовательность чисел Фибоначчи определяется следующим образом. Первое и второе числа Фибоначчи равны единице, а каждое следующее число Фибоначчи равно сумме двух предыдущих. Вот первые числа Фибоначчи: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, . . .
Во входном файле задано одно целое число N в диапазоне $1 \le N \le 2\,000$. Нужно вычислить и распечатать N-ое число Фибоначчи.
Поскольку это число может быть довольно большим (более 400 цифр), то вам необходимо собственноручно реализовать десятичную арифметику. Для этого рекомендуется использовать следующую структуру длинного числа:

```
typedef struct LongNum_s
{
    int len;      // сколько цифр в числе
    int arr[500]; // массив десятичных цифр числа
} LongNum;
```

Далее вам следует реализовать алгоритм сложения длинных чисел “в столбик”, как учили в школе, а также написать функцию распечатывания длинного числа. Тогда вы сможете вычислить нужное число Фибоначчи простым циклом по N.

Пример

input.txt	output.txt
12	144

Комментарий

Рекомендуется хранить цифры в массиве arr в обратном принятому у людей порядке: arr[0] — это единицы, arr[1] — десятки, а arr[len-1] — это ведущая цифра. Кстати, такой порядок “по-умному” называется little-endian.

Задача 6. Факториал

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Факториал от N (обозначается как $N!$) — это произведение всех целых чисел от 1 до N включительно.

Во входном файле задано одно целое число N в диапазоне $1 \leqslant N \leqslant 1\,000$. Нужно вычислить и распечатать факториал от N.

На этот раз вам понадобится обеспечить 3 000 цифр в структуре длинного числа, чтобы влез результат. Для вычисления ответа можно перебирать все k по порядку от 1 до N, и каждый раз домножать текущее длинное число на него.

Здесь нужно также реализовать умножение “в столбик”, однако можно полагать множитель одной цифрой (даже когда он больше десяти). В некотором смысле, эта операция называется умножением длинного числа (заданного массивом цифр) на короткое (заданное одним числом), и сама по себе она реализуется без вложенных циклов.

Пример

input.txt	output.txt
10	3628800

Задача 7. Сравнение асимптотик

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В курсе всё больше и больше будет акцент на улучшение асимптотического времени работы различных операций. При этом важно понимать, при каких изменениях асимптотическое время работы становится лучше (т.е. быстрее), а при каких хуже. В данной задаче предлагается реализовать сравнение для наиболее часто встречающихся асимптотик.

В данной задаче асимптотическое время работы задаётся как функция вида:

$T(N) = O(p^N \cdot N^s \cdot \log^l N)$

Здесь $p > 1$, $s > 0$ и $l > 0$ — произвольные вещественные числа. В этот класс попадают, например, асимптотики сортировки слиянием $O(N \log N)$, бинарного поиска $O(\log N)$, перебора всех N-битных чисел $O(2^N N)$.

Формат входного файла

В первой строке входного файла записано число Q — сколько тестовых случаев нужно обработать ($1 \leqslant Q \leqslant 10^5$). Далее идёт $2Q$ строк, каждая пара строк описывает один тестовый случай, то есть две асимптотики, которые надо сравнить.
Асимптотика в полном виде записывается как: "O(p^N N^s logN^l)" (без кавычек). В полном виде в ней три части, обязательно отделённые друг от друга и от окружающих скобок пробелом. Других пробелов нет. Части могут быть записаны в произвольном порядке. Вместо букв p, s и l в описании асимптотики записаны вещественные числа, задающие соответствующие коэффициенты. Все вещественные числа записаны с не более чем тремя знаками после десятичной точки, и лежат в пределах от 0 до 10 включительно. Кроме того, для коэффициента p верно: $p > 1$.
Кроме того, некоторые части могут быть опущены: в таком случае в произведении этой части нет. Если опущены все три части, то асимптотика будет записана в виде "O(1)" (без кавычек). Наконец, в компонентах N^s и logN^l может быть опущена степень: в таком случае она равна единице. Если степень опущена, то в описании отсутствует как вещественное число s или l, так и символ крышки непосредственно до него.
Замечание: рекомендуется использовать gets, strtok, sscanf и прочие стандартные функции для чтения асимптотики.

Формат выходного файла

В выходных данных должно быть ровно Q целых чисел, по одному числу в строке. Если в запросе первая асимптотика меньше второй, число должно быть равно −1. Если первая

асимптотика больше второй, то нужно вывести 1. Наконец, если они совпадают, то нужно вывести 0.

Пример

input.txt	output.txt
6 O(2^N N^3.5 logN^7.3) O(2^N N^4 logN^7.267) O(N^3.5 logN^7) O(logN^7.000 N^3.5) O(1) O(N^2) O(N^0.5) O(logN^7) O(2^N N) O(2^N) O(N logN) O(N^1.5)	-1 0 -1 1 1 -1

Задача 8. Числа со знаком

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Дана последовательность из N целых чисел. Некоторые из них заданы явно (как число в виде десятичной записи), а некоторые определяются как сумма, разность или произведение двух более ранних чисел последовательности. Нужно найти и вывести все N чисел, выполнив заданные арифметические операции.

Формат входного файла

В первой строке задано целое число N — количество чисел в последовательности ($1 \leqslant N \leqslant 1\,000$). В каждой из следующих N строк задано одно число. Числа нумеруются в порядке задания, начиная с нуля.

Описание k-ого числа начинается с цифры t, определяющей способ задания. Если $t = 0$, то дальше через пробел записано само число в явном виде.

Если $t = 1$, то данное число определяется как сумма, если $t = 2$ — то как разность, а если $t = 3$ — то как произведение. В любом из этих трёх случаев далее через пробел указано два целых числа a и b — номера членов последовательности, над которыми нужно выполнить арифметическую операцию, чтобы получить k-ое число ($0 \leqslant a, b < k$).

Гарантируется, что каждое число искомой последовательности по модулю меньше 10^{100}.

Формат выходного файла

Выведите информацию о каждом видео в строку таблицу. Первый столбец — это ID видео, второй — количество его фрагментов, а третий — суммарная длительность. Видео должны быть перечислены в таблице в порядке увеличения ID.

Примеры

input.txt	output.txt
9 0 12 0 15 3 0 1 0 135 2 3 2 2 0 1 3 5 4 2 0 0 1 4 6	12 15 180 135 -45 -3 135 0 90
5 0 100000000 3 0 0 3 1 1 0 1000000000000000000000000000000 2 3 2	100000000 10000000000000000 1000000000000000000000000000000000 1000000000000000000000000000000 -9900000000000000000000000000000000

Комментарий

Крайне рекомендуется использовать дополнительный код для задания отрицательных чисел, и соответственно использовать длинные числа фиксированной длины.

Задача 9. Выпуклый минимум

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Массив чисел $A_0, A_1, A_2, \ldots A_{n-1}$ называется выпуклым вверх, если:

$$\forall i < k < j : A_k < \frac{(j-k)A_i + (k-j)A_j}{(j-i)}$$

Дан выпуклый вверх массив A и коэффициент C. Требуется найти индекс элемента массива, на котором достигается минимум линейной функции:

$$\underset{i=0}{\overset{n-1}{argmin}}(A_i + C \cdot i) = ?$$

Если минимальное значение достигается на нескольких элементах массива, нужно найти номер первого такого элемента.

Формат входного файла

В первой строке записано одно целое число n — размер выпуклого массива ($1 \leqslant n \leqslant 10^5$). Далее записаны элементы массива A_i (n целых чисел, $|A_i| \leqslant 10^{15}$). Затем записано целое число q — количество запросов, которые нужно обработать ($1 \leqslant q \leqslant 10^5$). В остальных q строках записаны целые числа C_j, определяющие значения коэффициента линейной функции ($|C_j| \leqslant 10^9$).

Формат выходного файла

Требуется вывести q целых чисел: для каждого коэффициента C_j, записанного во входных данных, нужно вывести номер i первого элемента A_i, на котором достигается минимум ($A_i + C \cdot i$) при $C = C_j$.

Пример

input.txt	output.txt
10 9 4 0 -2 -2 -1 1 4 8 20 8 -5 1 -2 0 6 3 2 4	8 3 5 3 0 2 2 1

Пояснение к примеру

Рассмотрим коэффициент $C_2 = -2$. Выпишем значение соответствующей функции для всех элементов:

i=0: $9 - 2 * 0 = 9$

i=1: $4 - 2 * 1 = 2$

i=2: $0 - 2 * 2 = -4$

i=3: $-2 - 2 * 3 = -8$

i=4: $-2 - 2 * 4 = -10$

i=5: $-1 - 2 * 5 = -11$

i=6: $1 - 2 * 6 = -11$

i=7: $4 - 2 * 7 = -10$

i=8: $8 - 2 * 8 = -8$

i=9: $20 - 2 * 9 = 2$

Минимум достигается на двух элементах $i = 5$ и $i = 6$, и ответом является меньший номер $i = 5$.

Комментарий

Представьте себе, как бы вы решали задачу, если бы вместо массива A была дана гладкая функция $A(x)$, и нужно было бы найти минимум функции $(A(x) + Cx)$. Задача с массивом решается точно так же, нужно лишь найти дискретный аналог для понятия производной.

Задача 10. Шифр Шерлока Холмса

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Шерлок Холмс использовал различные способы шифрования. Один из них описан ниже. Дан текст из N строк, каждая из которых содержит N символов латинского алфавита. Также дан ключ — квадрат N×N клеток, в котором вырезано K клеток ($0 \le K \le \frac{N^2}{4}$). Ключ накладывается на текст, и через вырезанные клетки считывается первая часть расшифрованного сообщения. Затем ключ поворачивается на 90 градусов по часовой стрелке, и процесс повторяется. Всего ключ накладывается четыре раза (в ориентациях 0°, 90°, 180°, 270°). В каждой ориентации символы считываются по строкам сверху вниз, в каждой строке — слева направо, и добавляются в конец результирующей строки, изначально пустой. Ключ считается правильным, если при каждом повороте открываются только новые клетки, не пересекающиеся с клетками, открытыми в предыдущих ориентациях.

Формат входного файла

В первой строке входного файла записано целое число N ($1 \le N \le 250$). Далее находится описание ключа: в N строках записано по N цифр (0 или 1), разделенных пробелами (если 1 – клетка вырезана). Затем идет описание шифра: N текстовых строк, имеющих по N символов — букв латинского алфавита.

Формат входного файла

В выходной файл необходимо вывести сообщение «No», если во входных данных задан неправильный ключ. В противном случае необходимо вывести расшифрованный текст.

Пример

input.txt	output.txt
2 1 0 0 0 ab cd	abdc

Задача 11. Японский кроссворд

Источник:	космической сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Головоломка "Японский кроссворд", (также известная как Paint by Numbers — «рисование по номерам») представляет собой популярный вид логических задач. Кроссворд кодируется числами, указанными:

- слева от строк — для описания горизонтальных последовательностей;
- сверху над столбцами — для описания вертикальных последовательностей.

Каждое число обозначает длину группы чёрных клеток. Количество чисел указывает, сколько таких групп содержится в строке или столбце. Например, последовательность чисел 4 1 3 означает, что в ряду содержатся три группы: первая из 4 чёрных клеток, вторая из 1, третья из 3. В чёрно-белых кроссвордах группы должны быть разделены хотя бы одной пустой клеткой. В цветных — правило касается только одноцветных групп; разноцветные могут следовать подряд. Пустые клетки могут встречаться и по краям строки или столбца.
Задача: по заданным числовым описаниям определить возможное размещение групп клеток.

Формат входного файла

- В первой строке записано число строк $0 < X \leqslant 20$
- Далее следуют X строк с числами, задающими группы клеток для каждой строки.
- Затем идёт число столбцов $0 < Y \leqslant 20$.
- В следующих Y строках — группы клеток для каждого столбца.

Формат входного файла

Необходимо вывести одно или несколько решений в виде X строк длиной Y символов:

- закрашенная клетка — символ ' * '
- пустая клетка — символ ' . '

Если существует несколько решений, после каждого из них выводится:

- строка next — если это **не последнее** решение,
- строка end — если это **последнее** решение.

Если решений не существует, нужно вывести единственную строку: no solutions.

Примеры

input.txt	output.txt
2 1 1 1 1 2 1 1 1 1	*. .* next .* *. end
8 1 6 3 2 1 1 3 1 1 1 1 9 1 9 1 10 2 2 2 2 2 2 10 1 1 1 3 1 5 1 7 2 2 3 2 1 3 1 6 2 1 5 2 1 5 1 6	....****** ...**.*..* ...*..*..* .********* .********* ********** ..**...**. ..**...**. end